

《大数据与管理决策基础》

第 3 讲：SQL、KPI 与可审计指标层

课程讲义

目录

第 3 讲 SQL、KPI 与可审计指标层	3
3.1 学习目标	3
3.2 从数据模型到指标层	4
3.3 SQL 查询的关系语义	4
3.4 关系代数视角	5
3.5 事实粒度与分析粒度	6
3.6 连接、粒度与重复计数	6
3.7 KPI 的形式化口径	7
3.8 条件聚合与安全除法	8
3.9 空值、未完成事件与业务状态	9
3.10 窗口函数与经营节奏	9
3.11 从 SQL 到可审计指标层	10
3.12 从指标结果到管理解释	11
3.13 第 3 周实验案例	11
3.14 配套代码	12
3.15 可审计 SQL 的最低要求	12
习题	13
本讲小结	13
常见误区	14
参考文献	14

第 3 讲 SQL、KPI 与可审计指标层

本讲讨论如何把企业数据模型转化为可查询、可解释、可复核的管理指标。SQL 是关系数据之上的声明式查询语言，但在管理决策场景中，它不只是“取数工具”，而是连接业务对象、事实粒度、维度属性、时间窗口、分子分母和指标责任的表达语言。本讲先解释 SQL 查询的关系语义，再讨论连接、过滤、聚合、条件聚合和窗口函数如何形成销售额、毛利率、准时交付率、延期天数等 KPI；随后进一步说明比率型指标、事实表粒度、指标字典和可审计 SQL 的最低要求。课程目标不是让学生背诵 SQL 片段，而是训练学生写出能够进入管理汇报、数据产品和审计流程的指标查询。

课程接口

本讲把第二讲的数据模型推进到指标层。后续数据质量、统计看板、预测模型、因果评估和优化决策都需要以稳定指标为输入。若 SQL 指标口径不稳定，后续模型即使复杂，也只是在不可靠的数字上继续叠加复杂性。

3.1 学习目标

第一讲建立了从经营症状到行动闭环的主线，第二讲说明企业运营数据如何被建模为实体、事件、状态、事实表和维度表。第三讲进入更具体的问题：当数据模型已经存在时，如何用 SQL 从这些表中构造管理指标？

完成本讲后，学生应能够：

1. 解释 SQL 查询与关系模型中选择、投影、连接和聚合之间的关系；
2. 区分事务粒度、分析粒度和指标展示粒度；
3. 使用连接、分组聚合和条件聚合计算销售额、毛利、准时交付率和平均延期天数；
4. 正确处理比率型指标的分子和分母，避免直接平均百分比；
5. 说明空值、未完成事件和分母为零在管理解释中的含义；
6. 使用窗口函数计算排序、累计值、移动窗口和对象历史行为；
7. 设计包含名称、业务定义、对象范围、分子分母、时间窗口、单位、责任人与 SQL 片段的指标字典；
8. 写出可以被复核、被测试、被审计的指标查询。

3.2 从数据模型到指标层

SQL 常被误解为一种“取数工具”。在本课程中，SQL 的意义更深：它是把企业数据模型转化为共同事实基础的主要语言。看板上的销售额、经营分析报告中的延期率、预警规则中的高风险订单数，本质上都依赖某种可复核的查询表达。

第 2 讲强调事实表粒度。第 3 讲继续使用同一原则。若订单表的粒度是一行一个订单，那么订单数可以直接按行计数；若事实表的粒度是一行一个订单明细项，那么订单数需要按订单编号去重；若同一张表混入订单头、订单行和库存快照，就会导致重复计数。SQL 语法本身不会阻止这种错误，只有明确的业务粒度和指标定义才能防止它。

定义 3.1 (指标层). 指标层是组织对管理指标的统一表达层。它规定指标名称、业务定义、对象范围、事实粒度、分子分母、维度、时间窗口、单位、刷新频率、权限和责任人，并提供可复用的 SQL 或语义配置。

本讲把 SQL 放在如下链条中理解：

业务对象 → 事实粒度 → 查询表达 → 指标口径 → 管理解释 → 行动审计。

在管理场景中，一个 SQL 结果是否“正确”，不仅取决于语法是否能运行，还取决于它是否回答了正确的业务问题，是否使用稳定的数据范围，是否保留了分子分母，是否允许他人复核。

3.3 SQL 查询的关系语义

关系模型把数据看作关系，即带有属性名的元组集合。SQL 查询可以理解为在关系上的变换。尽管实际数据库会优化执行计划，但从逻辑上看，一个典型查询依次经历表来源、连接、过滤、分组、聚合、筛选和输出。

定义 3.2 (SQL 查询的逻辑处理顺序). 一个常见 SQL 查询的逻辑顺序可概括为：

FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

这一顺序表示查询语义的理解方式，不一定等于数据库系统实际执行计划。

例如，下面的查询按区域计算 2026 年 3 月订单收入：

代码 3.1: 区域销售额查询

```
SELECT
  c.region,
  SUM(o.quantity * p.unit_price) AS revenue
FROM orders AS o
JOIN customers AS c
  ON o.customer_id = c.customer_id
JOIN products AS p
```

```
ON o.product_id = p.product_id
WHERE o.order_date >= DATE '2026-03-01'
AND o.order_date < DATE '2026-04-01'
GROUP BY c.region
ORDER BY revenue DESC;
```

这段查询至少包含四个管理含义。第一，分析对象是 2026 年 3 月订单；第二，订单通过客户维度获得地区，通过产品维度获得售价；第三，收入按地区汇总；第四，输出排序用于比较区域经营表现。

评注 3.3. **WHERE** 在聚合前过滤原始事实记录，**HAVING** 在聚合后过滤分组结果。若把二者混淆，就可能把“筛选订单”和“筛选地区”混为一谈。前者改变事实集合，后者改变展示集合。

3.4 关系代数视角

SQL 之所以适合作为指标层语言，是因为它建立在关系模型和集合运算之上。虽然学生在实践中主要书写 SQL 语句，但理解其关系代数含义有助于判断查询是否改变了事实集合、是否丢失对象、是否重复计数。

表 3.1: 关系代数操作与 SQL 子句

关系操作	SQL 表达	管理含义
选择 σ	WHERE	限定事实集合，例如只保留某月订单。
投影 π	SELECT	选择输出属性，例如地区和收入。
连接 $\bowtie$	JOIN	补充业务对象属性，例如客户地区和产品价格。
分组聚合 γ	GROUP BY	将事实提升到分析粒度，例如区域级 KPI。
排序 τ	ORDER BY	为比较和展示建立顺序。

前一节的区域收入查询可以抽象为

$$\gamma_{\text{region}; \sum(q.\text{price}) \rightarrow \text{revenue}} (\sigma_{\text{March 2026}}(\text{orders} \bowtie \text{customers} \bowtie \text{products})).$$

这个表达式提醒我们，指标计算并不是随意拼接字段，而是在明确事实集合后做连接、选择和聚合。若事实集合错了，后续聚合再精细也无法恢复正确含义。

命题 3.4 (声明式查询与可审计性). *SQL* 的声明式特征使查询可以被复核：读者能够从查询中识别事实来源、过滤条件、连接键、分组维度和聚合方式。因此，清晰的 *SQL* 本身就是指标审计的一部分。

3.5 事实粒度与分析粒度

指标计算首先要声明粒度。粒度不清，是 KPI 错误的根源之一。

定义 3.5 (事务粒度、分析粒度与展示粒度). 事务粒度是源事实表中一行记录代表的业务事实；分析粒度是 SQL 查询分组后每一行结果代表的业务层级；展示粒度是看板、报告或管理会议中最终呈现的层级。

例如，订单明细事实表的事务粒度可能是一行一个订单明细项；按地区、月份聚合后的分析粒度是一行一个地区月份；在管理层月报中展示的粒度可能是一行一个区域。三者可以不同，但必须被明确说明。

表 3.2: 三类粒度的例子

粒度类型	示例	典型风险
事务粒度	一行一个订单明细项。	误把明细行数当订单数。
分析粒度	一行一个地区、月份。	聚合维度遗漏导致混合口径。
展示粒度	一行一个区域或业务线。	从低层百分比直接平均得到高层百分比。

SQL 查询写作的第一步不是输入 **SELECT**，而是写下如下句子：“本查询以什么事实表为基础？事实表一行代表什么？最终输出一行代表什么？”这句话比任何语法细节都更接近管理指标的核心。

3.6 连接、粒度与重复计数

连接是 SQL 中最容易产生管理分析错误的操作。内连接保留两张表中能够匹配的记录，左连接保留左表全部记录并补充右表信息，外连接可用于识别匹配与未匹配的边界。语法本身不难，真正困难的是判断连接是否改变了事实表粒度。

假设订单表 **orders** 的粒度是一行一个订单，产品表 **products** 的粒度是一行一个产品。如果 **products.product_id** 唯一，则

$$|\text{orders} \bowtie \text{products}| = |\text{orders}|.$$

这说明连接只是补充产品属性，没有复制订单事实。若产品表中同一个 **product_id** 出现两次，则连接后订单行数会膨胀，销售额、订单数和延期率都会被放大或扭曲。

表 3.3: 连接错误与指标后果

问题	数据原因	指标后果
事实行膨胀	维度键不唯一，或多对多关系未预聚合。	销售额、订单量、客户数被重复计算。

问题	数据原因	指标后果
事实行丢失	使用内连接，但维度缺失或外键无法匹配。	低估业务规模，遗漏异常对象。
时间错配	使用当前维度属性解释历史事实。	历史地区、等级、价格口径被改写。
粒度混合	订单头、订单行、库存快照混在同一聚合层。	KPI 分母不清，比较失真。

命题 3.6 (连接前后的行数检查). 若 F 是事实表, D 是维度表, 且 D 在连接键上唯一并覆盖 F 的外键, 则 F 与 D 的内连接不会改变事实行数。若连接后行数增加, 通常说明维度键不唯一; 若连接后行数减少, 通常说明存在无法匹配的外键。

因此, 在写任何管理指标 SQL 前, 应先回答三个问题: 事实表一行代表什么? 连接后的行数是否仍代表同一事实? 被连接的维度表在连接键上是否唯一?

3.7 KPI 的形式化口径

KPI 不是孤立数字, 而是一组被治理的定义。一个管理指标可以形式化为

$$M = (O, \mathcal{G}, \mathcal{T}, N, D, u, \rho),$$

其中 O 是被度量的对象集合或事件集合, $\mathcal{G}$ 是分组维度, $\mathcal{T}$ 是时间窗口, N 是分子或可加度量, D 是分母, u 是单位, ρ 是业务责任或解释责任。

以准时交付率为例。对象集合 O 是已交付订单, 时间窗口 $\mathcal{T}$ 可以是订单日期或承诺交付日期所在月份, 分子 N 是准时交付订单数, 分母 D 是已交付订单数, 单位 u 是百分比, 责任 ρ 可能涉及供应链、仓储和物流团队。

表 3.4: 样例 KPI 字典

指标	业务含义	计算口径
销售额	订单带来的标准收入。	$\sum q_i \cdot price_i$
毛利	收入扣除标准成本后的贡献。	$\sum q_i (price_i - cost_i)$
毛利率	收入中转化为毛利的比例。	$\sum GP_i / \sum Revenue_i$
准时交付率	已交付订单中按承诺日期交付的比例。	$\sum I(a_i \leq p_i) / n$
平均延期天数	订单超过承诺日期的平均延迟。	$\sum \max(a_i - p_i, 0) / n$

注意，毛利率和准时交付率是比率型指标。比率型指标通常不可加，不能直接把不同区域、产品或月份的百分比再取简单平均。正确做法是保留分子和分母，在目标粒度上重新计算：

$$\frac{\sum_g N_g}{\sum_g D_g} \neq \frac{1}{|G|} \sum_g \frac{N_g}{D_g}.$$

评注 3.7. 左边是按事实量加权后的总体比率，右边是分组百分比的平均数。除非每个分组分母相同，否则二者通常不同。

3.8 条件聚合与安全除法

SQL 中的 KPI 通常通过条件聚合表达。下面的查询计算区域级销售额、毛利、准时交付率与平均延期天数。

代码 3.2: 区域级运营 KPI 查询

```
WITH order_enriched AS (
  SELECT
    o.order_id,
    c.region,
    o.quantity * p.unit_price AS revenue,
    o.quantity * (p.unit_price - p.unit_cost) AS gross_profit,
    CASE
      WHEN o.actual_date <= o.promised_date THEN 1
      ELSE 0
    END AS on_time_flag,
    GREATEST(date_diff('day', o.promised_date, o.actual_date), 0) AS
      delay_days
  FROM orders AS o
  JOIN customers AS c
    ON o.customer_id = c.customer_id
  JOIN products AS p
    ON o.product_id = p.product_id
  WHERE o.actual_date IS NOT NULL
)
SELECT
  region,
  COUNT(*) AS delivered_orders,
  SUM(revenue) AS revenue,
  SUM(gross_profit) AS gross_profit,
  SUM(gross_profit) / NULLIF(SUM(revenue), 0) AS gross_margin,
  SUM(on_time_flag) * 1.0 / NULLIF(COUNT(*), 0) AS on_time_rate,
  AVG(delay_days) AS average_delay_days
FROM order_enriched
GROUP BY region
ORDER BY revenue DESC;
```

NULLIF 的作用是避免分母为零时产生错误或无意义结果。管理指标中，分母为零不是可以简单忽略的技术异常，而是需要解释的业务状态。例如某区域本月没有已

交付订单，则准时交付率并不是 0，而是没有定义；报表应区分“没有发生业务”和“业务发生但没有准时”。

定义 3.8 (安全除法). 安全除法是指在计算比率指标时显式处理分母为零、分母为空或分子分母口径不一致的情况。常见做法是返回空值、单独标记或在指标字典中说明不可定义状态。

3.9 空值、未完成事件与业务状态

SQL 中的 **NULL** 不等同于数值 0，也不等同于空字符串。它通常表示未知、缺失、尚未发生或不适用。在管理指标中，空值处理必须回到业务状态，而不能只按技术便利处理。

表 3.5: 空值与业务解释

情形	常见 SQL 处理	管理解释
未交付订单无实际日期	从准时率分母中排除，或单独形成未完成指标。	尚无交付结果，不应简单计为延期或准时。
区域没有已交付订单	比率返回 NULL 。	指标不可定义，而不是表现为 0。
产品成本缺失	标记数据质量问题。	毛利无法可信计算，需要补数或剔除说明。
客户维度缺失	左连接保留事实并标为未知客户。	业务规模存在，但归属和分层解释受限。

评注 3.9. 一个严谨的指标系统应区分三类状态：“业务没有发生”“业务发生但结果失败”“数据缺失导致无法判断”。这三者在管理上对应完全不同的行动。

3.10 窗口函数与经营节奏

聚合查询会把多行压缩为一行。窗口函数则不同，它在保留行身份的同时，对与当前行相关的一组行进行计算。这个思想非常适合经营分析，因为管理者既需要看到每个订单，也需要知道它在客户历史、区域排名或时间趋势中的位置。

代码 3.3: 客户最近三笔订单收入

```
SELECT
  o.customer_id,
  o.order_id,
  o.order_date,
  o.quantity * p.unit_price AS revenue,
  SUM(o.quantity * p.unit_price) OVER (
    PARTITION BY o.customer_id
    ORDER BY o.order_date, o.order_id
```

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS customer_recent_revenue,
ROW_NUMBER() OVER (
  PARTITION BY o.customer_id
  ORDER BY o.order_date, o.order_id
) AS customer_order_sequence
FROM orders AS o
JOIN products AS p
  ON o.product_id = p.product_id
ORDER BY o.customer_id, o.order_date, o.order_id;
```

窗口函数在管理分析中常用于：

- 对客户、产品、供应商或区域进行排名；
- 计算累计销售额、累计延期订单数和累计缺货次数；
- 计算移动窗口收入、近三次订单金额或近七天投诉数；
- 识别同一对象的前后状态变化，例如订单状态流转或库存变化。

窗口函数的风险在于窗口定义不清。若 **ORDER BY** 没有唯一排序键，排名和前后项可能不稳定；若窗口框架没有明确说明行数或时间范围，累计值可能被误解为移动值；若分区维度错误，客户级行为会被混成全局行为。

3.11 从 SQL 到可审计指标层

在小型课堂实验中，学生可以把 SQL 写在脚本里。但在企业环境中，指标需要被治理。所谓可审计指标层，是指组织能够回答以下问题：这个指标由谁定义？它使用哪些源表？它的事实粒度是什么？它能按哪些维度下钻？它是否可以跨时间求和？它的 SQL 是否经过测试？当一个经营会议引用该指标时，其他团队能否复现同样结果？

一个课程级指标配置可以写成如下结构：

代码 3.4: 指标配置示例

```
metrics:
- name: gross_margin
  label: Gross margin
  type: ratio
  numerator: gross_profit
  denominator: revenue
  grain: order
  time_dimension: order_date
  dimensions: [region, segment, category, channel]
  owner: finance
```

指标层的核心价值并不只是减少重复代码。它让组织能够围绕同一事实讨论问题，并让模型、看板、报告和审计共用一套经过治理的定义。

表 3.6: 可审计指标层的最低字段

字段	说明
指标名称	组织中统一使用的名称。
业务定义	指标回答的管理问题。
事实粒度	计算依据的一行事实代表什么。
分子与分母	比率指标必须保存分子和分母定义。
时间窗口	使用订单日期、交付日期、快照日期还是入库日期。
维度范围	可以按哪些对象、组织层级和时间层级下钻。
责任人与版本	谁维护口径，何时修改，如何复核。

3.12 从指标结果到管理解释

SQL 生成的是结果表，管理者需要的是解释和行动。一个指标结果进入经营会议前，至少应完成六步解释。

表 3.7: KPI 结果解释模板

解释环节	需要回答的问题
事实描述	指标在什么对象、时间窗口和维度上发生了什么变化？
比较基准	与上期、目标值、预算、同类区域或历史分位数相比如何？
口径说明	分子、分母、排除条件、空值和未完成事件如何处理？
异常定位	哪些客户、产品、渠道、供应商或流程节点贡献最大？
行动建议	可以采取哪些行动，行动对象和约束是什么？
复核路径	SQL、输入表版本、指标字典和责任入能否被追溯？

例如，若南区准时交付率下降，合格的解释不应只写“南区表现较差”。更好的写法是：以已交付订单为分母，南区 2026 年 3 月准时交付率低于全体水平；下降主要来自高优先级订单和某一产品类别；未交付订单未进入分母；下一步应检查供应商交期、仓库出库时延和物流路线，并对高价值高风险订单建立预警。这样的解释才把 SQL 输出连接回管理行动。

3.13 第 3 周实验案例

本周样例数据位于 `data/sample/`。表结构如下：

- `customers.csv`: 客户维度，一行一个客户；
- `products.csv`: 产品维度，一行一个产品，包含标准售价和单位成本；
- `orders.csv`: 订单事实，一行一个订单；
- `inventory.csv`: 库存快照，一行一个产品、仓库、日期。

课堂实验分为四步：

1. 加载 CSV 数据并检查行数、主键唯一性和外键匹配；
2. 用 SQL 连接订单、客户和产品三张表，构造订单级收入、毛利和交付状态；
3. 按区域、客户分层、产品类别和渠道计算 KPI；
4. 将 KPI 口径写入 `metrics.yml`，并用 Python 复核关键指标。

学生需要特别注意，`orders.csv` 中的销售额不是原始字段，而是由数量和产品价格计算得到；毛利也不是原始字段，而是由数量、售价和成本共同确定。这种设计是刻意的，因为管理分析经常需要把事实表与维度表连接后才能得到指标度量。

3.14 配套代码

第三周 Python 复核代码位于：

`src/bdm_decision/cases/week03_sql_kpi.py`

核心函数包括：

- `enrich_orders()`：连接订单、客户和产品，生成订单级收入、毛利和交付状态；
- `summarize_by()`：在指定分组维度上计算订单数、收入、毛利率、准时交付率和延期天数；
- `rolling_customer_revenue()`：复现窗口函数思想，计算客户最近三笔订单收入。

运行方式：

代码 3.5: 运行第三周案例代码

```
PYTHONPATH=src python3 src/bdm_decision/cases/week03_sql_kpi.py
```

配套 SQL 位于：

`src/bdm_decision/warehouse/build_kpi_tables.sql`

该脚本以 DuckDB 风格读取 CSV，建立 `order_enriched`、`kpi_by_region`、`kpi_by_month` 和 `customer_order_window` 等视图，便于学生把讲义中的 SQL 逻辑与可运行实验对应起来。

3.15 可审计 SQL 的最低要求

可审计 SQL 并不意味着复杂，而意味着别人可以理解、复现和质疑。课程要求学生写在写 KPI SQL 时至少满足以下规范：

1. 每个查询说明事实表粒度；
2. 每个连接说明连接键及其唯一性假设；

3. 每个比率指标保留分子和分母；
4. 每个时间窗口说明采用订单日期、交付日期还是快照日期；
5. 对空值、未交付订单和分母为零的情况给出处理规则；
6. 查询输出字段命名稳定，能够进入后续 dashboard 或报告；
7. 至少提供一个行数检查或 Python 复核结果。

这些规范也为第 4 讲数据质量奠定基础。数据质量不是在指标算完之后才出现的问题；相反，重复主键、缺失外键、异常日期和字段空值会直接改变 SQL 的输出。因此，SQL/KPI 与数据质量不是先后无关的两个模块，而是同一条指标可信度链条上的两个环节。

习题

习题 3.10. 使用 `data/sample/orders.csv`、`customers.csv` 和 `products.csv`，按客户分层计算销售额、毛利率和准时交付率。要求写出 SQL，并说明事实表粒度、分组粒度、分子、分母和时间窗口。

习题 3.11. 解释为什么不能直接平均各区域毛利率来得到总体毛利率。请构造一个两区域的小例子，给出错误算法和正确算法的数值差异。

习题 3.12. 写出一个 SQL 查询，找出每个客户最近三笔订单的累计收入。请说明 `PARTITION BY`、`ORDER BY` 和窗口框架分别承担什么作用。

习题 3.13. 对 `orders` 与 `products` 的连接结果做行数检查。若连接后行数增加，说明可能出现了什么问题？若连接后行数减少，又可能是什么原因？

习题 3.14. 为“高优先级订单延期率”设计一条指标字典记录，至少包含对象范围、事实粒度、分子、分母、时间窗口、单位、维度和责任部门。

本讲小结

本讲建立了 SQL 与管理指标之间的联系。SQL 的核心不是机械语法，而是关系数据上的声明式表达；KPI 的核心不是单个数字，而是对象范围、粒度、分子分母、时间窗口、单位和责任的组合。连接决定事实是否被复制或遗漏，聚合决定指标落在哪个业务层级，空值处理决定指标能否被正确解释，窗口函数决定经营节奏如何被表达，指标层决定同一个组织是否能围绕同一口径讨论问题。

进入下一讲后，课程将继续追问：即使 SQL 口径正确，输入数据是否可信？主键是否重复？外键是否缺失？日期是否异常？指标口径是否被不同部门改写？这些问题构成数据质量与指标治理的核心。

常见误区

1. 把 SQL 运行成功等同于指标正确。语法正确只是最低要求，指标还需要业务口径正确。
2. 把百分比当作可加度量。比率型指标应在目标粒度重新聚合分子和分母。
3. 在没有检查维度键唯一性的情况下连接维度表，导致事实重复。
4. 把未交付订单直接计入准时交付率分母，混淆“尚无结果”和“结果失败”。
5. 把 NULL 当作 0，掩盖业务未发生、数据缺失或不适用之间的差异。
6. 只保存最终 KPI 数字，不保存生成该数字的 SQL、时间窗口和输入数据版本。
7. 在窗口函数中缺少稳定排序键，导致排名、前后项或移动窗口结果不可复现。

参考文献

- [1] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
- [2] Chamberlin, D. D., and Boyce, R. F. (1974). SEQUEL: A structured English query language. *Proceedings of the 1974 ACM SIGFIDET Workshop on Data Description, Access and Control*, 249–264.
- [3] Date, C. J. (2003). *An Introduction to Database Systems*. Addison-Wesley.
- [4] Kimball, R., and Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley.
- [5] International Organization for Standardization and International Electrotechnical Commission. (2016). *ISO/IEC 9075: Information technology – Database languages – SQL*.
- [6] Few, S. (2006). *Information Dashboard Design*. O’Reilly Media.
- [7] Karwin, B. (2010). *SQL Antipatterns: Avoiding the Pitfalls of Database Programming*. Pragmatic Bookshelf.